RECLAIMAI

Smart Lost-and-Found Matching Platform

Software Specification

Version 1.0 | 30 October 2026 | Draft baseline for implementation and testing

Purpose and intended outcome Build a web application that allows people to report lost and found items, discover explainable potential matches, submit ownership claims, and record successful returns. This specification defines observable behavior and verification criteria for a focused end-to-end release using React, FastAPI, and PostgreSQL.

Source requirement

The selected project is ReclaimAI: Smart Lost-and-Found Matching Platform. Development follows specification-driven development: specification -> plan -> implement -> test -> review. The initial technology baseline is React for the web interface, FastAPI for the REST API, and PostgreSQL for persistent storage.

How to read this specification

Requirements marked B are baseline requirements agreed for this project concept. Requirements marked P are proposed elaborations needed to make the system implementable and testable. “Shall” indicates required behavior. Requirement IDs connect implementation tasks, API contracts, acceptance criteria, and tests. The word “AI” in the product name refers to smart matching behavior; Release 1 uses an explainable deterministic scoring engine rather than a trained machine-learning model.

Document map

Section	Coverage
1-2	Scope, assumptions, actors and access
3-5	Functional requirements, lifecycle, screens and journeys
6-8	Data model, matching design, REST API and errors
9-10	Quality requirements, acceptance criteria and tests
11-12	Delivery plan, traceability, decisions and future scope

1 Scope and baseline assumptions

Included in the initial release

Users create lost or found reports, browse active reports, receive ranked potential matches, submit claims, and track recovery status. Administrators review claims, manage categories, moderate reports, and confirm successful returns. Matching is performed server-side and every displayed score includes a human-readable explanation.

ID	Basis	Requirement
B01	B	Users shall be able to create lost-item reports.
B02	B	Users shall be able to create found-item reports.
B03	B	Users shall be able to browse, search and filter public report data.
B04	B	The system shall compare each eligible report with reports of the opposite type.
B05	B	Potential matches shall be ranked by an explainable score from 0 to 100.
B06	B	A user shall be able to submit an ownership claim for a potential match.
B07	B	An administrator shall be able to approve or reject claims and record return completion.
B08	B	The solution shall expose a React web interface, FastAPI REST API and PostgreSQL database.
B09	B	Required fields, status rules and privacy rules shall be enforced by the API.
B10	B	Verification shall include valid, invalid, duplicate and unauthorized operations.

Baseline assumptions

A01 - Release 1 serves a single organization, campus, venue or community. Multi-tenant separation is outside the release.

A02 - Accounts use email and password authentication implemented by the application for learning purposes. Roles are USER and ADMIN. Passwords are stored only as secure hashes.

A03 - Any authenticated user may create lost and found reports. A user may edit or withdraw only reports created by that user while those reports remain active.

A04 - Public report data intentionally excludes contact details, private verification questions, private verification answers and precise storage instructions.

A05 - Images are represented by an optional image URL in Release 1. Direct file upload and cloud storage are outside scope.

A06 - “Smart matching” means deterministic weighted matching using structured attributes and normalized text. No external AI service is required.

A07 - All dates and times are stored in UTC. Event dates are displayed in the browser timezone.

A08 - Email, SMS, push notifications and real-time chat are not required. Match and claim updates are visible in the application.

Excluded from Release 1

Image recognition, OCR, maps and GPS distance, real-time messaging, social login, payments, public comments, external email/SMS, mobile applications, multilingual search, facial recognition, government integrations, cross-organization tenancy, machine-learning training, and automatic proof-of-ownership decisions are excluded. The system assists matching and claim review; it does not autonomously establish legal ownership.

2 Actors, authentication and access control

P01 - Every operation involving private report, claim, user or administrative data shall require an authenticated active account. The API derives user identity and role from a validated JWT access token. Client-supplied owner IDs or roles shall never determine access.

Capability	User	Administrator
Register and sign in	Yes	Yes
Create lost/found report	Yes, for self	Yes, for self
Browse public active reports	Yes	Yes
Read full report	Own reports; public projection of others	All reports
Edit/withdraw report	Own active reports	Any report
View potential matches	Own reports only	All reports
Submit claim	Yes, except against own found report	Yes
Review claim	No	Approve or reject
Confirm return	Own involved report after approved claim	Any approved claim
Manage categories	No	Yes
View dashboard	Personal	System-wide
Delete audit history	No	No

Authorization rules

P02 - Report ownership is the immutable reporter_id written at creation. Users shall never receive another user’s email, phone number, verification answer, private notes or exact storage location through public list or detail endpoints.

P03 - Missing, invalid or expired identity returns HTTP 401. Authenticated users attempting a role-restricted action receive 403. Requests for a nonexistent resource or another user’s private resource return the same 404 response where necessary to avoid revealing existence.

P04 - Administrators may inspect full report and claim details for moderation and return verification. Administrative access shall not permit password hash exposure. Audit events remain immutable.

Primary journeys

Lost-item owner: register/sign in -> create lost report -> view ranked found-item matches -> open match explanation -> submit claim -> track review -> arrange verification outside the public UI -> see item recorded as recovered.

Finder: register/sign in -> create found report -> review matching lost reports -> see claims associated with the found item -> after approval and handover, see item recorded as returned.

Administrator: sign in -> inspect active reports and pending claims -> compare private verification details -> approve or reject claim -> confirm completed return -> view updated recovery statistics.

3 Functional requirements

Account and session management

FR01 [P] - Registration shall accept full name, unique email and password. On success, the API creates an active USER account and returns a session token or requires sign-in according to the implementation plan. Duplicate normalized email returns 409.

FR02 [P] - Sign-in shall validate email and password and return a signed access token. Error messages shall not reveal whether an email exists. Inactive accounts cannot sign in.

Create and manage reports

FR03 [B01, B02] - On valid submission, the server shall create one report with an immutable UUID, readable reference, authenticated reporter, report type LOST or FOUND, status ACTIVE, server timestamps and version 1. The response returns the saved private report projection.

FR04 [B09] - Required fields shall pass Section 6 rules. Missing, blank, invalid enum, impossible date or unsupported fields shall fail without creating a report, match or audit record.

FR05 [B03] - Public browse shall display only active, non-private fields. It shall support report type, category, colour, brand, area, date range and text query filters, plus pagination and newest-first ordering.

FR06 [P] - An owner may edit descriptive report attributes while status is ACTIVE or POTENTIAL_MATCH_FOUND. Changes shall increment version, create an audit event and trigger recalculation of affected matches. Report type and reporter are immutable.

FR07 [P] - An owner may withdraw an unresolved report. Withdrawal changes status to CLOSED, deactivates open matches, and blocks new claims. Physical deletion is not provided.

Matching

FR08 [B04, B05] - After a report is created or materially edited, the API shall compare it with eligible active reports of the opposite type. The same lost-found pair shall have at most one current match record.

FR09 [B05] - Each match shall store component scores, total score, classification and calculation version. Results for an owned report shall be ordered by total score descending, then most recently calculated.

FR10 [B05] - Match responses shall explain positive, partial and missing indicators without exposing private fields. Scores at or above 80 are STRONG, 60-79 are POSSIBLE, 40-59 are WEAK, and below 40 are not shown by default.

Claims and recovery

FR11 [B06] - An authenticated user may submit one active claim for a specific lost-found pair. The claim shall include an ownership explanation and answer to the found report’s private verification question. Claimant contact data comes from the authenticated profile.

FR12 [B07] - An administrator shall change a claim from PENDING to UNDER_REVIEW, APPROVED or REJECTED according to Section 4. Rejection requires a review comment; approval requires a review comment confirming that evidence was checked.

FR13 [B07] - For an APPROVED claim, an administrator or an involved report owner may record handover completion. The claim becomes COMPLETED, the lost report becomes RECOVERED, the found report becomes RETURNED, and competing pending claims are rejected as ITEM_UNAVAILABLE in one transaction.

FR14 [P] - Dashboards shall display counts computed from persisted data. Personal dashboard: active reports, potential matches, pending claims and recovered items. Admin dashboard: active lost/found reports, strong matches, pending claims, completed returns and category distribution.

Auditability

FR15 [P] - Report creation, edits, status changes, match recalculation, claim submission, claim review and return completion shall create append-only audit events containing actor, action, resource, timestamp and non-sensitive changed fields.

4 Lifecycles and business rules

Report lifecycle

From	To	Condition
New submission	ACTIVE	Server controlled.
ACTIVE	POTENTIAL_MATCH_FOUND	At least one visible match has score >=60.
POTENTIAL_MATCH_FOUND	CLAIM_UNDER_REVIEW	At least one claim moves to UNDER_REVIEW.
ACTIVE / POTENTIAL_MATCH_FOUND	CLOSED	Owner or admin withdraws before completed return.
Any unresolved status	EXPIRED	Admin action or scheduled policy after configured age.
CLAIM_UNDER_REVIEW	RECOVERED (lost)	Approved claim handover completed.
CLAIM_UNDER_REVIEW	RETURNED (found)	Approved claim handover completed.

BR01 - Terminal report statuses are CLOSED, EXPIRED, RECOVERED and RETURNED. Terminal reports are read-only except for administrator notes. Terminal reports do not participate in new matching or claims.

BR02 - POTENTIAL_MATCH_FOUND and CLAIM_UNDER_REVIEW are derived from current eligible matches and claims. Display status shall remain consistent with persisted match and claim state.

Claim lifecycle

From	To	Condition
New submission	PENDING	Server controlled.
PENDING	UNDER_REVIEW	Administrator begins review.
PENDING / UNDER_REVIEW	APPROVED	Administrator review comment required; both reports available.
PENDING / UNDER_REVIEW	REJECTED	Administrator review comment required.
PENDING	CANCELLED	Claimant cancels before review.
APPROVED	COMPLETED	Handover confirmed.
APPROVED	REVOKED	Administrator records failed handover or new evidence; reason required.

BR03 - All unlisted transitions return 409 INVALID_TRANSITION. A claim cannot be submitted for reports with the same type, the same reporter on both sides, terminal statuses, or an already completed pair.

BR04 - Approval does not automatically complete the return. Approval indicates that the claim may proceed to handover; COMPLETED records that the item was returned.

BR05 - A user may not answer the verification question for a report authored by that same account. The finder’s stored verification answer is never returned to the claimant.

Atomicity and duplicate handling

BR06 - Report creation, audit event creation and initial match calculation shall commit consistently. If synchronous matching fails, report creation may commit with matching_status=PENDING only if the failure is recorded and a retry path exists; the Release 1 preferred implementation performs both in one transaction for simplicity.

BR07 - Claim approval, report status updates, rejection of competing claims and audit events shall commit atomically. If any required write fails, all writes roll back.

BR08 - The UI disables repeat submission while a request is pending. Create-report and create-claim APIs accept an Idempotency-Key. Exact replay returns the original result; reuse with different content returns 409.

5 Web interface specification

Screen	Required content and behavior
Register / Sign in	Accessible labelled fields, validation, generic sign-in failure, session persistence and sign-out.
Home	Product purpose, two primary actions: Report Lost Item and Report Found Item; public impact counters may show aggregate non-sensitive statistics.
Create Report	Report-type-specific multi-section form, required labels, category, item attributes, event date/time, area, public description, private verification question/answer, contact confirmation and review step.
Browse Reports	Active public reports, lost/found toggle, search, filters, sorting, pagination, empty state and report cards.
Report Details	Public fields for others; owner view additionally shows private management controls, match count and claim status. Never show private answer.
Potential Matches	Owned report summary, ranked cards, percentage, classification, explanation, filter threshold and claim action.
Submit Claim	Ownership explanation, private verification answer, privacy warning, review and submit.
My Activity	Tabs for My Reports, My Claims and Matches; status badges and links.
Admin Dashboard	System counts, pending claims, strong matches, recent activity and category management.
Claim Review	Side-by-side lost/found details, private evidence, audit trail, review comment and valid actions only.

Form and interaction behavior

UI01 - Required inputs shall be visibly marked. Client validation mirrors server rules but the API remains authoritative. User input remains available after validation or network errors.

UI02 - Report creation shall show the readable reference and matching state only after server confirmation. A successful creation navigates to the report detail page.

UI03 - Match percentages must always be accompanied by classification and explanation. Colour shall not be the sole indicator.

UI04 - Search and filter state shall be reflected in the URL query string where practical. “Clear filters” returns to the default active-report view.

UI05 - Loading, empty, success and error states shall be explicit. Errors shall not expose stack traces, SQL or secrets. Mutation buttons remain disabled during active submission.

UI06 - At 375 px and 1440 px widths, primary actions shall remain usable without page-level horizontal scrolling. Keyboard navigation, visible focus, labels and readable validation are required.

UI07 - A 401 response returns the user to sign in. A 403 explains that the action is unavailable. A 404 shows “Report not found or unavailable.” A 409 status conflict prompts refresh.

Reference user journey

1. Create lost report for a black Dell laptop bag lost near Block B cafeteria.
2. Create found report for a black Dell shoulder bag found one day later in Block B cafeteria.
3. Open the lost report and calculate/display a STRONG match with an explanation.
4. Submit a claim with private ownership evidence.
5. Administrator reviews both reports and approves the claim.
6. Handover is confirmed; reports become RECOVERED and RETURNED and dashboard totals update.

6 Data model and validation

All identifiers are UUIDs unless stated otherwise. Text lengths are measured after trimming. User-entered text is stored as plain text and escaped on display. Timestamps use ISO 8601 UTC. Database constraints shall reinforce API validation.

User

Field	Type and rule
id	UUID primary key.
full_name	Required string, 2-100 characters.
email	Required normalized lowercase email, unique, max 254.
password_hash	Required secure hash; never returned by API.
role	Enum USER or ADMIN; default USER.
is_active	Boolean; default true.
created_at / updated_at	Server timestamps.

Category

Field	Type and rule
id	UUID primary key.
name	Required unique string, 2-60 characters.
description	Optional string, max 300.
is_active	Boolean; inactive categories cannot be used for new reports.
created_at	Server timestamp.

ItemReport

Field	Type and rule
id / reference	UUID / unique readable reference RA- plus at least six digits; never reused.
report_type	Enum LOST or FOUND; immutable.

Field	Type and rule
reporter_id	Required User FK; immutable.
item_name	Required string, 2-120.
category_id	Required active Category FK.
colour	Required string, 2-40.
brand	Optional string, max 80.
event_date	Required date; not later than current local date + 1 day.
approximate_time	Optional local time.
area	Required string, 2-120; broad location safe for public display.
location_details	Optional private string, max 300.
description	Required public string, 10-1500.
distinguishing_features	Optional public-safe string, max 800.
verification_question	Required for FOUND; optional for LOST; 10-300.
verification_answer_hash	Required for FOUND; normalized answer hash or protected value; never returned.
storage_instructions	Optional private string, max 300; FOUND only.
image_url	Optional valid http/https URL, max 500.
status	ReportStatus enum; server controlled.
matching_status	Enum PENDING, COMPLETE, FAILED.
version	Positive integer starting at 1.
created_at / updated_at	Server timestamps.

ItemMatch

Field	Type and rule
id	UUID primary key.
lost_report_id / found_report_id	Required FKs; unique pair; types enforced by service.
component scores	Non-negative integers within configured maxima.
total_score	Integer 0-100.
match_level	STRONG, POSSIBLE, WEAK or HIDDEN.
explanation	Structured JSON generated from non-sensitive factors.
algorithm_version	Required string, e.g. rules-v1.
is_active	Boolean; false when either report becomes ineligible.
calculated_at	Server timestamp.

Claim and AuditEvent

Record	Required fields
Claim	id, lost_report_id, found_report_id, claimant_id, ownership_explanation (20-1500),

Record	Required fields
	verification_answer_hash/protected value, status, review_comment, reviewer_id, timestamps, version. One nonterminal claim per claimant and pair.
AuditEvent	id, actor_id nullable for system, action, resource_type, resource_id, occurred_at, changed_fields JSON excluding secrets, request_id. Append-only.

Indexes and constraints

Index active reports by report_type, status, category_id, event_date and created_at. Index normalized colour, brand and area where supported. Index matches by lost_report_id, found_report_id, total_score and is_active. Index claims by status, claimant_id and report pair. Enforce foreign keys, unique email, unique references and pair uniqueness. No cascade delete shall remove historical claims or audit events.

7 Smart matching specification

The matching engine is deterministic, explainable and versioned. It produces candidate rankings, not ownership decisions. Only active reports of opposite type and matching category are eligible by default.

Weighted scoring model - rules-v1

Factor	Maximum	Rule summary
Category	25	Exact category required for full score; different category normally makes pair ineligible.
Area/location	20	Normalized exact or token overlap on public area. Private location details are not exposed.
Date proximity	15	Same day 15; 1-2 days 12; 3-5 days 8; 6-10 days 4; otherwise 0. Found date before lost date scores 0.
Colour	10	Normalized exact 10; recognized multi-colour overlap 5; otherwise 0.
Brand	10	Both blank 0; normalized exact 10; partial token match 5; otherwise 0.
Item name	10	Token similarity scaled 0-10.
Description/features	10	Keyword overlap across public description and distinguishing features scaled 0-10.
Total	100	Sum of component scores, capped at 100.

Normalization

- Convert comparison text to lowercase.
- Trim leading/trailing whitespace and collapse repeated spaces.
- Remove punctuation for token comparison while preserving stored original text.
- Map configured synonyms, for example “mobile” and “phone”, without changing source content.
- Remove a small documented stop-word set for description similarity.
- Do not include private verification answers, contact details or storage instructions in matching.

Classification and visibility

Score	Level	Default behavior
80-100	STRONG	Displayed prominently.
60-79	POSSIBLE	Displayed in standard results.
40-59	WEAK	Hidden by default; owner may reveal weak matches.
0-39	HIDDEN	Stored only if needed for diagnostics; not shown to users.

Matching rules

ME01 - Recalculation occurs on creation and when category, item name, colour, brand, event date, area, description or distinguishing features change. The current algorithm version overwrites component scores for the unique pair and updates calculated_at.

ME02 - Candidate generation shall exclude same-type reports, the same report, terminal reports, reports from the same owner on both sides, and found reports dated more than one day before the lost date.

ME03 - The explanation shall be generated from component results, for example “Same category”, “Same colour”, “Found one day later”, or “Brand not provided”. It shall not reproduce hidden private values.

ME04 - Changing scoring weights requires a new algorithm_version and regression tests. Existing match records may be recalculated by an administrator operation or migration script.

Worked example

Factor	Observation	Score
Category	Bag = Bag	25/25
Area	“Block B Cafeteria” vs “Cafeteria, Block B”	20/20
Date	Found one day after lost	12/15
Colour	Black = Black	10/10
Brand	Dell = Dell	10/10
Item name	Laptop bag vs laptop shoulder bag	8/10
Description	Shared black, grey strap, front scratch keywords	6/10
Total	Strong match	91/100

8 REST API contract

Base path: /api/v1. Requests and responses use application/json. Protected endpoints require Authorization: Bearer <token>. Lists return {items, page, page_size, total}. Default page_size is 20 and maximum is 100. Unknown request fields are rejected.

Method and path	Contract
POST /auth/register	Create USER account. Body: full_name, email, password. Returns 201 public user.
POST /auth/login	Body: email, password. Returns 200 access token and user projection.

Method and path	Contract
GET /users/me	Return current public profile.
GET /categories	Return active categories; authenticated.
POST /categories	ADMIN only; create category. Returns 201.
PATCH /categories/{id}	ADMIN only; update name, description or active state.
POST /reports	Create for current user. Idempotency-Key required. Returns 201 with Location.
GET /reports	Public-safe active list. Filters: type, category, colour, brand, area, from_date, to_date, q, page, page_size.
GET /reports/{id}	Return owner/admin private projection or public-safe projection.
PATCH /reports/{id}	Owner or ADMIN; active report only. If-Match version required.
POST /reports/{id}/withdraw	Owner or ADMIN; reason required. Returns closed report.
GET /reports/{id}/matches	Owner or ADMIN only. Optional minimum_score; ranked results.
POST /reports/{id}/matches/recalculate	Owner or ADMIN; recalculates eligible pairs. Returns summary.
POST /claims	Create claim. Body: lost_report_id, found_report_id, ownership_explanation, verification_answer. Idempotency-Key required.
GET /claims	Current user claims; ADMIN may use scope=all and status filter.
GET /claims/{id}	Claimant, involved report owners or ADMIN with projection rules.
PATCH /claims/{id}/status	ADMIN for review transitions; claimant may CANCEL pending claim. If-Match required.
POST /claims/{id}/complete	ADMIN or involved report owner; approved claim only.
GET /dashboard/me	Personal summary.
GET /dashboard/admin	ADMIN-only system summary.

Filtering and ordering

API01 - q is a case-insensitive token search over item_name, brand, area and public description, limited to 100 trimmed characters. Filters combine with AND. Default report ordering is created_at descending then id descending. Match ordering is total_score descending then calculated_at descending.

Response projections

API02 - PublicReport excludes reporter email/phone, location_details, storage_instructions, verification question/answer, private notes and audit details. PrivateReport may include the verification question for a claimant only at claim-entry time, but never includes the stored verification answer. AdminReport includes review-required fields but never includes password hashes.

Example request

POST /api/v1/reports

```
{
  "report_type": "LOST",
  "item_name": "Dell laptop bag",
  "category_id": "<uuid>",
  "colour": "Black",
  "brand": "Dell",
  "event_date": "2026-10-28",
  "area": "Block B Cafeteria",
  "description": "Black shoulder bag with a grey strap and front pocket."
}
```

9 Errors, security and operational requirements

Error contract

All errors use {"error":{"code":"...","message":"...","fields":{},"request_id":"..."}}. fields maps invalid field names to safe messages and is empty for non-validation errors.

HTTP	Code and meaning
400	MALFORMED_JSON - body cannot be parsed.
401	UNAUTHENTICATED - missing, invalid or expired token.
403	FORBIDDEN - role or ownership disallows action.
404	NOT_FOUND - resource absent or unavailable to caller.
409	INVALID_TRANSITION, DUPLICATE_RESOURCE or IDEMPOTENCY_CONFLICT.
412 / 428	VERSION_CONFLICT / PRECONDITION_REQUIRED.
422	VALIDATION_ERROR - invalid fields, filters, enums or business input.
429	RATE_LIMITED - too many attempts where configured.
500 / 503	INTERNAL_ERROR / UNAVAILABLE - no internal details returned.

Quality requirements

ID	Requirement and verification
NF01 Security	Hash passwords with an approved adaptive algorithm. Sign tokens with secrets outside source control. Validate authorization and input on every request. Use SQLAlchemy parameterization. Escape rendered text.
NF02 Privacy	Never expose contact data, verification answers, exact storage instructions or private review notes in public responses, logs or match explanations.
NF03 Performance	In a stated local/test environment with 10,000 reports and 50 concurrent users, p95 list/detail reads <=1.5 s and report/claim writes <=2.5 s, excluding client network latency.
NF04 Matching	For up to 5,000 eligible opposite-type reports in the test dataset, candidate scoring completes <=3 s or uses a documented background/retry path. Ranking is deterministic for identical data and algorithm version.
NF05 Usability	Primary flows work at 375 px and 1440 px, with keyboard access, visible focus, labels and non-colour status indicators.
NF06 Durability	Successful records survive restart. Use migrations and documented backup/restore. No partial claim completion state.
NF07 Observability	Log request ID, route, status, duration and safe error category. Do not log passwords, tokens, contact data, verification answers or private location.
NF08 Compatibility	Support current stable Chrome and Edge for the training/demo environment.
NF09 Verification	Automate core API validation, authorization, matching, lifecycle and privacy tests. Execute frontend smoke tests for the main journey.
NF10 Maintainability	Separate routes, schemas, models, services and matching logic. Scoring weights are defined in one documented

ID	Requirement and verification
	configuration/module.

Architecture boundary

The React frontend presents forms and results. The FastAPI service owns authentication, validation, authorization, workflow, matching and transactions. PostgreSQL persists business records. The browser shall not calculate authoritative scores or set protected statuses. Development, test and production configurations use separate environment values; secrets remain outside source control.

10 Acceptance criteria and test cases

Tests use User A, User B and Administrator accounts plus an inactive account. Verify API response, visible UI behavior and persisted state. Matching tests use fixed data so expected component scores are exact.

Test	Given and action	Expected result	Trace
TC01	Register with valid name, unique email and valid password.	201; USER account created; password absent from response.	FR01
TC02	Register duplicate email or invalid fields.	409 or 422; no duplicate account.	FR01
TC03	Sign in with valid and invalid credentials.	Valid returns token; invalid returns generic 401.	FR02
TC04	Create valid LOST report.	201; RA reference; ACTIVE; owner from token; version 1.	FR03
TC05	Create valid FOUND report with verification question/answer.	201; private answer not returned or logged.	FR03, NF02
TC06	Submit blank name, short description, future date, unknown field.	422 with field errors; no report/match/audit remnants.	FR04
TC07	User B reads User A report.	Only public projection; no contact/private fields.	P02, API02
TC08	User B attempts to edit or withdraw User A report.	404 or 403 as contracted; state unchanged.	P03, FR06
TC09	Browse with type/category/area/date/q filters.	Correct active results, total and ordering.	FR05, API01
TC10	Edit a material field with correct version.	200; version increments; audit written; matches recalculated.	FR06, ME01
TC11	Edit using stale or missing If-Match.	412 or 428; state unchanged.	API contract
TC12	Match sample Dell bag reports.	Stored total 91, STRONG, component explanation correct.	FR08-FR10
TC13	Compare LOST with LOST or terminal report.	Pair excluded; no active match.	ME02
TC14	Private answer contains matching words.	Score unchanged; explanation does not expose answer.	ME03, NF02
TC15	Create report with no candidate >=60.	No displayed default match; correct status.	FR10
TC16	Submit valid claim for eligible pair.	201 PENDING; claimant from token; private answer hidden.	FR11
TC17	Submit duplicate active claim or claim own found report.	409/422; one active claim only.	BR03
TC18	Non-admin attempts approval.	403; claim and reports unchanged.	FR12
TC19	Admin rejects without comment, then with comment.	First 422; second 200 REJECTED.	FR12

Test	Given and action	Expected result	Trace
TC20	Admin approves claim for available pair.	APPROVED; review event written; reports not yet terminal.	FR12, BR04
TC21	Complete approved handover.	Claim COMPLETED; lost RECOVERED; found RETURNED; competitors rejected atomically.	FR13, BR07
TC22	Attempt new claim after item returned.	409 INVALID_TRANSITION or ITEM_UNAVAILABLE.	BR01, BR03
TC23	Withdraw active report.	CLOSED; matches inactive; new claim blocked.	FR07
TC24	Replay report/claim creation with same idempotency key/body.	Original response; no duplicate. Different body returns 409.	BR08
TC25	Inject database failure during completion.	All claim/report/audit changes roll back.	BR07, NF06
TC26	Enter script-like text in public fields.	Stored as plain text and rendered inert.	NF01
TC27	Use keyboard and narrow viewport through report-to-claim flow.	All primary actions, labels, focus and errors usable.	UI06, NF05
TC28	Run stated data-volume test.	Performance/matching targets met with recorded environment.	NF03, NF04
TC29	Deactivate user and attempt sign-in.	401; historical records retained.	FR02, P04
TC30	Inspect application logs after private-field operations.	No secrets, contact details, private answer or storage instructions.	NF02, NF07

Overall release acceptance

All baseline requirements and adopted proposed requirements shall trace to passing tests. The complete lost-report -> found-report -> strong-match -> claim -> approval -> completed-return journey shall pass end to end. No unresolved defect may expose private verification data, allow unauthorized mutation, create duplicate records within the retry window, lose committed data or permit an invalid lifecycle transition. Test evidence shall record build identifier, environment, execution date and result.

11 Delivery plan and implementation sequence

The project shall be implemented from this specification in small verified increments. Each task ends with database/API/UI checks before moving forward.

Task	Implementation outcome	Primary verification
1. Workspace and configuration	VS Code workspace, React/Vite frontend, FastAPI backend, PostgreSQL database, environment files, CORS and health endpoint.	Frontend loads; API health returns 200; DB connection succeeds.
2. Database foundation	SQLAlchemy models, enums, Alembic migrations, default categories and indexes.	Migrations apply/rollback; seed query returns categories.
3. Authentication	Registration, login, password hashing, JWT dependency and role checks.	Auth tests including inactive and duplicate users.
4. Report API	Create/list/detail/edit/withdraw endpoints, projections, validation and audit events.	Swagger and pytest CRUD/privacy tests.
5. Matching service	Normalization, component scorers, candidate selection, persistence and explanations.	Unit tests for every boundary and 91-point sample.
6. Claims workflow	Claim creation, admin review, completion transaction and competing-claim handling.	Lifecycle and rollback tests.

Task	Implementation outcome	Primary verification
7. React shell	Routing, navigation, API client, auth context, protected routes, reusable styles/components.	Login/logout and route protection.
8. Report UI	Home, forms, browse, details, filters and My Activity.	Create/edit/search flows against real API.
9. Match and claim UI	Ranked match cards, explanation, claim form and status tracking.	End-to-end owned-report journey.
10. Admin UI	Dashboard, pending claim review, category management and completion action.	Admin role and full recovery workflow.
11. Hardening	Error states, responsive layout, accessibility, logging, migrations and sample data.	Acceptance tests TC01-TC30.
12. Demo and review	Scripted sample scenario, README, screenshots, known limitations and future scope.	Fresh setup from README and final demonstration.

Definition of done

- Code is implemented in the agreed React, FastAPI and PostgreSQL stack.
- Database can be created from migrations and seeded without manual table creation.
- All mandatory endpoints are documented in FastAPI OpenAPI/Swagger.
- The frontend completes the reference journey without direct database manipulation.
- Matching results are deterministic, versioned and explained.
- Private fields are absent from public responses and logs.
- Mandatory acceptance tests pass and evidence is recorded.
- README contains prerequisites, environment variables and exact start commands.

12 Traceability, decisions and future scope

Requirement traceability summary

Capability	Requirement IDs	Primary implementation area	Acceptance tests
Authentication and roles	FR01-FR02, P01-P04	auth router/service, user model, protected routes	TC01-TC03, TC08, TC18, TC29
Report management	FR03-FR07	report router/service, report forms and lists	TC04-TC11, TC23
Smart matching	FR08-FR10, ME01-ME04	matching_service.py and match views	TC12-TC15, TC28
Claims and recovery	FR11-FR13, BR03-BR08	claim service, admin review UI	TC16-TC25
Dashboard and audit	FR14-FR15	dashboard queries, audit model	TC20-TC21, TC30
Security and usability	NF01-NF10, UI01-UI07	cross-cutting frontend/backend	TC05, TC07, TC26-TC30

Decisions requiring review before implementation

Decision	Proposed baseline	Effect if changed
Authentication	Local email/password + JWT for training project.	External identity provider changes setup and user model.
Image handling	Optional external image URL only.	Direct uploads require storage, validation and deletion rules.
Verification answer storage	Protected comparison value; never public.	Supporting fuzzy/private review may

Decision	Proposed baseline	Effect if changed
	Prefer salted hash where exact normalized comparison is sufficient.	require encrypted storage and stricter access controls.
Matching threshold	60 displayed by default; 80 strong.	Changes UX, report derived status and regression expectations.
Report expiry	Manual/admin for Release 1; optional policy later.	Scheduled expiry requires background job.
Claim reviewer	Administrator only.	Finder-led approval changes permissions and dispute handling.
Deployment	Local development first; environment-ready configuration.	Cloud deployment adds hosting, domains, HTTPS and backup ownership.
Retention	No physical deletion in UI; administrative policy pending.	Production use requires retention and deletion procedures.

Future enhancements

- Image upload and image-similarity assistance.
- Map coordinates and distance-aware matching.
- Email/SMS notifications and secure finder-owner messaging.
- Organization tenancy for campuses, airports, events and residential communities.
- Synonym expansion and multilingual matching.
- Machine-learning reranking trained only after suitable consented data, evaluation and bias review.
- Duplicate-report detection, moderation queues and abuse controls.
- Progressive web application and push notifications.

Specification change control

Store this specification with the source code. For a requirement change, update the requirement, affected data/API contract, acceptance criteria and tests before changing implementation. Record version, date, reason and affected IDs. Generated code or a tool prompt does not override this specification.

Implementation baseline statement

Ready for development This Version 1.0 draft is detailed enough to begin the planned end-to-end VS Code implementation. The first implementation step is workspace setup and PostgreSQL configuration, followed by database migrations, authentication, reports, matching, claims, React pages and acceptance testing in the order defined in Section 11.